

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	35% of CIA		
CO2 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	S.N.V.S Karthik	Reg. No.:	192424186
Section / Batch:	CSE-AI/2024	Date:	11-08-26

Code of Conduct

I _____ S.N.V.S Karthik (192424186) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ____Karthik_____

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Part of Speech Tagging	Morphological decomposition of connected, connecting, and connection; identification of roots, suffixes, inflectional vs. derivational forms, and normalization for document indexing.	Q1
English Word Classes, Tagsets for English, Rule-Based Part of Speech Tagging	Morphological parsing of unhappy, happiness, and happily; identification of prefixes, suffixes, base forms, and semantic effects of derivational morphology.	Q2
Counting Words, Part of Speech Tagging, English Word Classes	Stemming and root extraction for played, player, and playing; handling grammatical variations and word formation changes for search preprocessing.	Q3
Finite-State Morphological Analysis, Rule-Based Part of Speech Tagging, Transformation-Based Tagging	Finite-state parsing of writes, writing, and written; modeling regular and irregular verb inflections and root normalization.	Q4
English Word Classes, Rule-Based Part of Speech Tagging, Counting Words	Porter Stemmer-based processing of relational, relation, and relate; application of derivational suffix removal and stem extraction for retrieval.	Q5

Annexure A – ANALYTICAL PROBLEM SOLVING

1. Given the input words: “connected”, “connecting”, “connection”, design a morphological analysis pipeline for a document indexing system.

- Apply rules to decompose each word into root and suffix components.
- Differentiate between inflectional and derivational forms within the dataset.
- Normalize all variants to a common base form for efficient retrieval.
- Determine the parsed structure and normalized output for each word.
- Write a Python program to implement the above morphological analysis pipeline. The program should accept the given input words, perform decomposition, classify each suffix as inflectional or derivational, normalize the words to their common base form, and display the parsed structure and normalized output in a tabular format.

2. Given the input words: “unhappy”, “happiness”, “happily”, design a morphological parsing module for sentiment analysis.

- Identify prefixes, suffixes, and base forms using rule-based decomposition.
- Ensure semantic consistency across derived word forms.
- Classify each transformation as inflectional or derivational.
- Produce the root and morphological breakdown for each word.
- Write a Python program to implement the above morphological parsing module. The program should accept the given input words, identify the prefixes, suffixes, and base forms, classify each transformation as inflectional or derivational, and display the morphological breakdown and normalized root word in a tabular format.

3. Given the input words: “played”, “player”, “playing”, design a stemming-based preprocessing module for a search engine.

- Apply morphological rules to strip affixes and identify the base form.
- Handle both grammatical variations and word formation changes.
- Ensure consistent root extraction across all forms.
- Determine the stem and classification for each input word.
- Develop a Python program for a stemming-based preprocessing module used in a search engine. The program should process the input words "played", "player", "playing" by applying appropriate stemming rules to remove prefixes/suffixes where applicable, identify the stem of each word, distinguish between grammatical inflections and word-formation changes, and generate a structured output showing the original word, extracted stem, removed affix (if any), transformation type (inflectional/derivational), and the final normalized form suitable for indexing and information retrieval.

4. Given the input words: “writes”, “writing”, “written”, design a finite-state morphological parsing module for a text processing system.

- Apply state transitions to represent suffix transformations and irregular forms in verb inflection.
- Differentiate between regular and irregular inflectional patterns within the dataset.
- Ensure consistent mapping of all forms to a common root representation.
- Determine the state transitions, morphological breakdown, and normalized output for each word.
- Develop a Python program that implements a finite-state morphological parser for the input words "writes", "writing", "written". The program should construct a finite-state transition model to process regular suffixes and irregular verb forms, identify the corresponding root word, classify each input as a regular or irregular inflection, trace the

sequence of state transitions during parsing, and display the state transition path, morphological components, root form, and normalized representation for each input word in a well-formatted output.

5. Given the input words: “relational”, “relation”, “relate”, design a Porter Stemmer-based preprocessing module for document retrieval.

- Apply stemming rules step-by-step to remove derivational suffixes and obtain base forms.
- Handle variations in suffix patterns while preserving semantic consistency.
- Ensure uniform root extraction across all derived forms.
- Determine the stemming steps, intermediate forms, and final stem for each input word.
- Develop a Python program that implements the Porter Stemming algorithm for the input words "relational", "relation", "relate". The program should apply the Porter stemming rules sequentially to remove derivational suffixes, display the intermediate form obtained after each stemming step, handle different suffix patterns while preserving the word meaning, and generate the final stem for each word. The output should clearly present the original word, applied stemming rule(s), intermediate forms, and the final normalized stem in a structured format suitable for document retrieval applications.

Rubrics for Calculation

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Q. No. / Criteria Level	Problem Understanding	Method Selection	Solution Process	Accuracy of Calculation	Interpretation of Results	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. Morphological Analysis of “connected”, “connecting”, and “connection”

Aim

To design a morphological analysis pipeline for a document indexing system by decomposing the given words, identifying roots and suffixes, classifying the suffixes, and normalizing all variants to a common base form. The question requires the identification of inflectional and derivational morphology and implementation using Python.

Morphological Analysis

Word	Root/Base	Suffix	Type	Normalized Form
connected	connect	-ed	Inflectional	connect
connecting	connect	-ing	Inflectional	connect
connection	connect	-ion	Derivational	connect

Python Program

```
words = ["connected", "connecting", "connection"]
for word in words:
    if word.endswith("ed"):
        root = word[:-2]
        suffix = "-ed"
        morph_type = "Inflectional"
    elif word.endswith("ing"):
        root = word[:-3]
        suffix = "-ing"
        morph_type = "Inflectional"
    elif word.endswith("ion"):
        root = word[:-3]
        suffix = "-ion"
        morph_type = "Derivational"
    else:
        root = word
        suffix = "None"
        morph_type = "None"
    print(word, root, suffix, morph_type, root)
```

Result

Original Word	Parsed Structure	Type	Normalized Output
connected	connect + ed	Inflectional	connect
connecting	connect + ing	Inflectional	connect
connection	connect + ion	Derivational	connect

Q2. Morphological Parsing of “unhappy”, “happiness”, and

“happily”

Aim

To design a rule-based morphological parsing module for sentiment analysis by identifying prefixes, suffixes, base forms, and the semantic effect of derivational morphology.

Morphological Analysis

Word	Prefix	Base	Suffix	Type	Normalized Root
unhappy	un-	happy	—	Derivational	happy
happiness	—	happy	-ness	Derivational	happy
happily	—	happy	-ly	Derivational	happy

Semantic Interpretation

- **unhappy:** The prefix **un-** reverses the meaning of “happy”, producing a negative sentiment.
 - **happiness:** The suffix **-ness** converts the adjective “happy” into the noun “happiness”.
 - **happily:** The suffix **-ly** converts the adjective “happy” into the adverb “happily”.
- All three words are normalized to the base form **happy**.

Python Program

```
words = ["unhappy", "happiness", "happily"]
for word in words:
    if word.startswith("un"):
        prefix = "un-"
        base = word[2:]
        suffix = "-"
    elif word.endswith("ness"):
        prefix = "-"
        base = word[:-4]
        suffix = "-ness"
    elif word.endswith("ly"):
        prefix = "-"
        base = word[:-2]
        suffix = "-ly"
    else:
        prefix = "-"
        base = word
        suffix = "-"
    print(word, prefix, base, suffix, "Derivational", base)
```

Result

Word	Morphological Breakdown	Type	Normalized Root
unhappy	un- + happy	Derivational	happy
happiness	happy + -ness	Derivational	happy

Q3. Stemming of “played”, “player”, and “playing”

Aim

To design a stemming-based preprocessing module for a search engine that extracts a common stem and distinguishes grammatical inflections from word-formation changes.

Stemming Analysis

Word	Stem	Removed Affix	Transformation Type	Normalized Form
played	play	-ed	Inflectional	play
player	play	-er	Derivational	play
playing	play	-ing	Inflectional	play

Python Program

```
words = ["played", "player", "playing"]
for word in words:
    if word.endswith("ed"):
        stem = word[:-2]
        affix = "-ed"
        morph_type = "Inflectional"
    elif word.endswith("ing"):
        stem = word[:-3]
        affix = "-ing"
        morph_type = "Inflectional"
    elif word.endswith("er"):
        stem = word[:-2]
        affix = "-er"
        morph_type = "Derivational"
    else:
        stem = word
        affix = "None"
        morph_type = "None"
    print(word, stem, affix, morph_type, stem)
```

Result

Original	Extracted Stem	Removed Affix	Type	Final Form
played	play	-ed	Inflectional	play
player	play	-er	Derivational	play
playing	play	-ing	Inflectional	play

Q4. Finite-State Morphological Parsing of “writes”, “writing”, and “written”

Aim

To design a finite-state morphological parser that represents regular and irregular verb inflections and maps all forms to the common root **write**.

Morphological Analysis

Word	Morphological Structure	Pattern	Root	Normalized Form
writes	write + s	Regular inflection	write	write
writing	write + ing	Regular inflection with spelling change	write	write
written	irregular past participle of write	Irregular inflection	write	write

Finite-State Model

The finite-state model can be represented as:

```
q0 --write--> q1
q1 --s-----> q2 → writes
q1 --ing----> q3 → writing
q1 --irregular--> q4 → written
State interpretation:
```

- **q0**: Start state
- **q1**: Root word “write” recognized
- **q2**: Regular third-person suffix **-s** recognized
- **q3**: Progressive suffix **-ing** recognized
- **q4**: Irregular participle form **written** recognized
- Final states represent successfully parsed words.

State Transition Table

Input	State Path	Type	Root	Normalized Form
writes	q0 → q1 → q2	Regular	write	write
writing	q0 → q1 → q3	Regular	write	write
written	q0 → q1 → q4	Irregular	write	write

Python Program

```
words = ["writes", "writing", "written"]
transitions = {
    "write": {
        "s": "q2",
        "ing": "q3"
    }
}
```

```

},
"written": {
    "irregular": "q4"
}
}

```

for word in words:

```

if word == "writes":
    root = "write"
    path = "q0 -> q1 -> q2"
    form = "Regular Inflection"
    component = "write + s"
elif word == "writing":
    root = "write"
    path = "q0 -> q1 -> q3"
    form = "Regular Inflection"
    component = "write + ing"
elif word == "written":
    root = "write"
    path = "q0 -> q1 -> q4"
    form = "Irregular Inflection"
    component = "irregular form"

```

```

print(word, path, component, form, root)

```

Result

The three forms are normalized to the common root:

```

\boxed{write}

```

Q5. Porter Stemmer Processing of “relational”, “relation”, and “relate”

Aim

To apply Porter stemming rules sequentially to remove derivational suffixes and obtain a common normalized stem for document retrieval.

Stemming Process

1. relational

The suffix **-ational** is replaced by **-ate**:

```

[relational \rightarrow relate]

```

Then the suffix **-ate** is removed:

```

[relate \rightarrow rel]

```

Final stem:

```

\boxed{rel}

```

2. relation

The suffix **-ation** is replaced by **-ate**:

```

[relation \rightarrow relate]

```

Then **-ate** is removed:

[relate \rightarrow rel]

Final stem:

[boxed{rel}]

3. relate

The suffix **-ate** is removed:

[relate \rightarrow rel]

Final stem:

[boxed{rel}]

Summary Table

Original Word	Step 1	Step 2	Final Stem
relational	relational → relate	relate → rel	rel
relation	relation → relate	relate → rel	rel
relate	—	relate → rel	rel

Python Program

```
words = ["relational", "relation", "relate"]
for word in words:
    original = word
    steps = []
    if word.endswith("ational"):
        word = word[:-6] + "ate"
        steps.append(word)
    elif word.endswith("ation"):
        word = word[:-5] + "ate"
        steps.append(word)
    if word.endswith("ate"):
        word = word[:-3]
        steps.append(word)
    print("Original:", original)
    print("Intermediate forms:", steps)
    print("Final stem:", word)
    print()
```

Final Result

Original Word	Applied Rule	Intermediate Form	Final Stem
relational	-ational → -ate, -ate → ∅	relate → rel	rel
relation	-ation → -ate, -ate → ∅	relate → rel	rel
relate	-ate → ∅	rel	rel

Overall Conclusion

The five morphological processing tasks demonstrate the use of rule-based decomposition, inflectional and

derivational analysis, stemming, finite-state morphological parsing, and Porter stemming. The normalized forms are **connect** for Q1, **happy** for Q2, **play** for Q3, **write** for Q4, and **rel** for Q5. These normalized representations reduce morphological variation and improve consistency in document indexing, sentiment analysis, search preprocessing, and information retrieval.